INF6103 : Cybersécurité des Infrastructures Critiques

TP 2 : Scénario d'intrusion contre les systèmes de contrôle industriels

---

Rapport de test d'intrusion

Équipe 6

Frederic Boileau	1716129
Tristan Moreau	2554828
Michael Panov	2194637

Polytechnique Montréal

Automne 2026 — 6 novembre 2026

Table des matières

1 Reconnaissance **2**
1.1 Cartographie des machines et services 2
1.2 Architecture réseau et modèle de Purdue 2
1.3 Accès à la page d'authentification 5
1.4 Tentative d'authentification 6
1.5 Brute force 7

2 Capture du traffic **8**
2.1 Description 8
2.2 MiTM 10
2.3 Capture et analyse de la communication 10
2.4 Lecture avec mbpoll 13
2.5 Analyse du traffic mbpoll 14

3 Injections **15**
3.1 Arrêter le PLC interface 15
3.2 Arrêter le PLC commande 16
3.3 Modification des valeurs de l'interface de contrôle 18
3.4 Analyse des composants 19
3.5 Expérimentation avec différentes valeurs 20
3.6 Explosion 21

4 Defense **21**

1 Reconnaissance

1.1 Cartographie des machines et services

Question 1.1 — *Identifiez les machines présentes sur le réseau et les services qu'elles offrent (tableau : adresse IP, nom d'hôte, ports ouverts, services).*

La découverte a été réalisée depuis la RasPi (`192.168.95.122`) avec `nmap -sV -p- -T4 192.168.95.0/24`. Onze hôtes actifs ont été identifiés (tableau 1).

TABLE 1 – Machines et services détectés sur le réseau `192.168.95.0/24`

Adresse IP	Nom d'hôte	Port ouvert		Service détecté (version)
`192.168.95.2`	`plc.scada`	502	tcp	Modbus/TCP
		8080	tcp	HTTP — Node.js (Express)
`192.168.95.5`	`ihm.scada`	8009	tcp	AJP13 — Apache JServ (protocole v1.3)
		9090	tcp	HTTP — Apache Tomcat/Coyote JSP 1.1
`192.168.95.10`–`.15`[a]	—	502	tcp	Modbus/TCP (`mbap?`)
		55555	tcp	Inconnu
`192.168.95.101`[a]	`simulation.scada`	80	tcp	HTTP — Apache httpd 2.4.18 (Ubuntu)
		55555	tcp	Inconnu
`192.168.95.254`	`inf6103-tp2`	22	tcp	SSH — OpenSSH 8.2p1 (Ubuntu)
		80	tcp	HTTP — Apache httpd 2.4.18 (Ubuntu)
		2222	tcp	SSH — OpenSSH 8.4p1 (Debian)
		8080	tcp	HTTP — Node.js (Express)
		9090	tcp	HTTP — Apache Tomcat/Coyote JSP 1.1
`192.168.95.122`	`2e66cc4cb86d`[b]	22	tcp	SSH — OpenSSH 8.4p1 (Debian)

[a] Les adresses `.10` à `.15` et `.101` partagent la même adresse MAC (`02:42:C0:A8:5F:65`) : il s'agit d'une seule machine possédant plusieurs adresses IP.
[b] Notre point d'entrée (RasPi).

1.2 Architecture réseau et modèle de Purdue

Question 1.2 — *Construisez une représentation de l'architecture du réseau industriel. Positionnez les actifs observés dans les niveaux appropriés du modèle de Purdue. Justifiez chaque positionnement à partir d'éléments observables.*

La figure 1 présente l'architecture du réseau industriel reconstituée à partir de la reconnaissance, avec chaque actif positionné dans le niveau correspondant du modèle de Purdue.

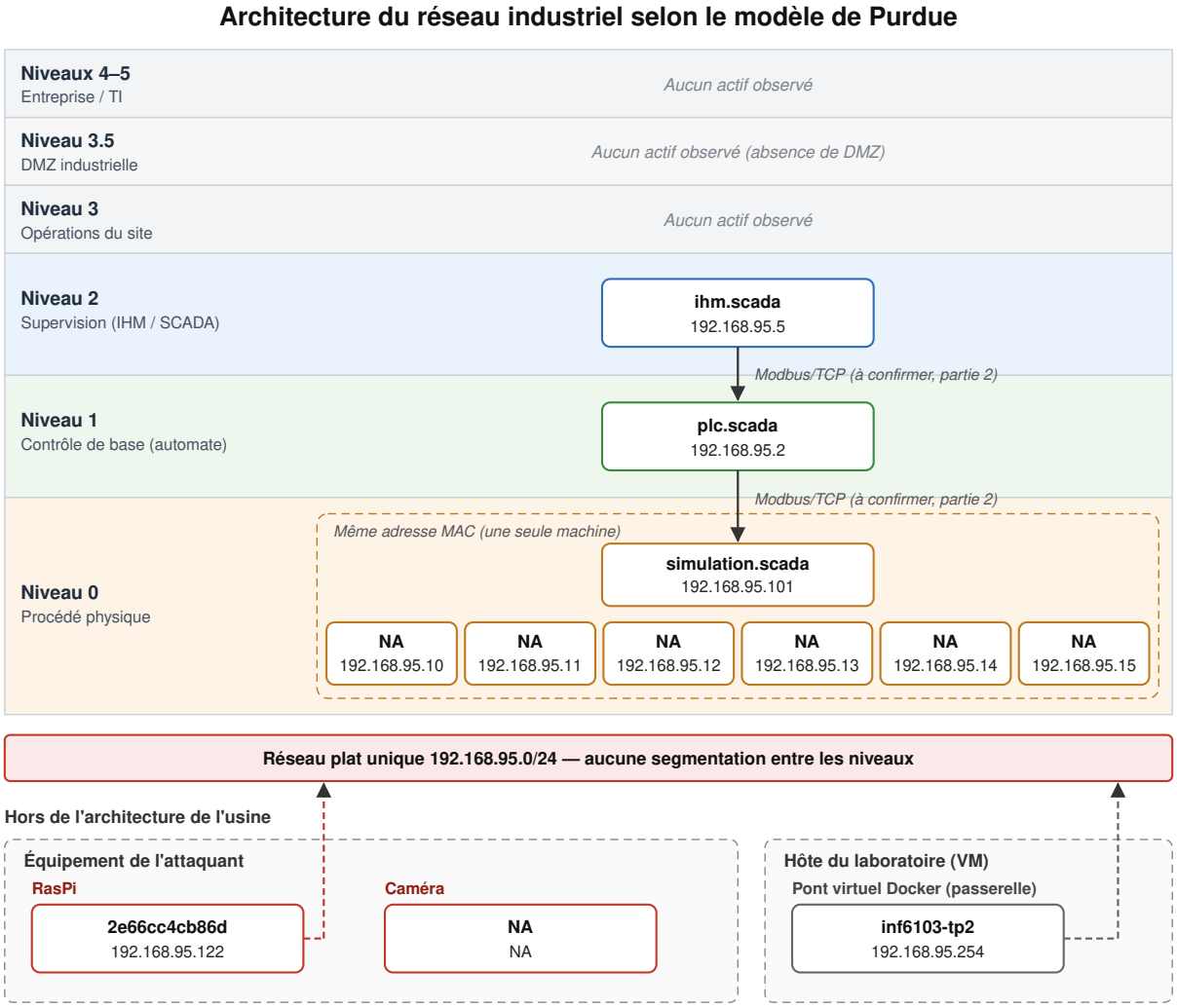


FIGURE 1 – Architecture du réseau industriel selon le modèle de Purdue (note : réalisé en svg par Claude selon notre spec)

Le positionnment sur l'échelle de Purdue repose sur trois types d'informations observées :

1. Les protocoles exposés
2. Les noms d'hôte (scan `nmap -Sv`)
3. Les adresses MAC relevées lors du scan

Niveau 0 - Procédé physique : simulation.scada (192.168.95.101) et 192.168.95.10-15 Les six adresses exposent un serveur Modbus/TCP (port 502) et rien d'autre d'après notre scan. Elles partagent toutes la même adresse MAC. Le nom d'hôte décrit explicitement le rôle de simulation de processus physique.

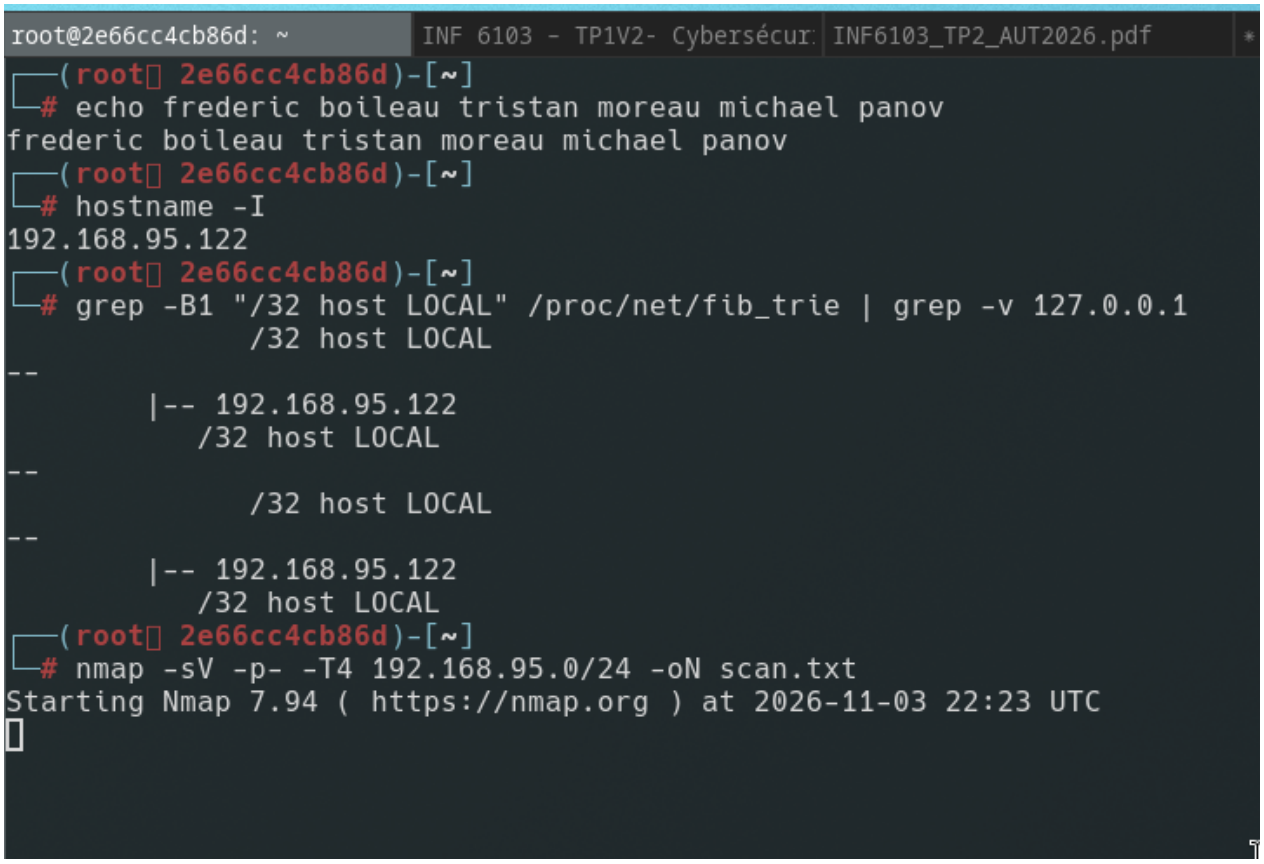
Niveau 1 - Contrôle de base : plc.scada (192.168.95.2) L'hôte expose un serveur Modbus/TCP, un protocole par lequel un PLC peut être lu et commandé par un superviseur. Il expose par contre aussi une application Node.js/Express au port 8080. De plus il public la même bannière que le port 8080 publié par la VM décrit dans l'énoncé comme une interface de contrôle de PLC. Finalement son nom d'hôte indique explicitement qu'il sagirait d'un PLC. Sous toute réserve on peut poser

l'hypothèse qu'il sert de lien entre le niveau 2 de supervision et les processus physiques et leurs équipements (actuators, sensors) au niveau 0.

Niveau 2 - Supervision : ihm.scada (192.168.95.5) Cet hôte ne publie aucun port de type Modbus contrairement aux précédents. Il publie cependant le port 9090 qui selon nmap serait un serveur Apache Tomcat/Coyote. Il présente d'ailleurs la même bannière que le port 9090 publié par la VM décrit dans l'énoncé comme l'interface de contrôle du système industriel. Il y aussi le port 8009 (AJP13), probablemen une erreur de configuration puisque ce dernier est destiné à la connexion interne avec Tomcat.

Niveau 3-4-5 : Nous n'avons rien observé à ces niveaux.

Hors de l'architecture il y a la carte RasPi et la caméra qui sont des machines de l'attaquant et ne prennent donc pas place dans le modèle de Purdue.



1.3 Accès à la page d'authentification

Question 1.3 — *Accédez à la page d'authentification de l'interface de contrôle du système industriel.*

Trouvée en faisant un guess éduqué

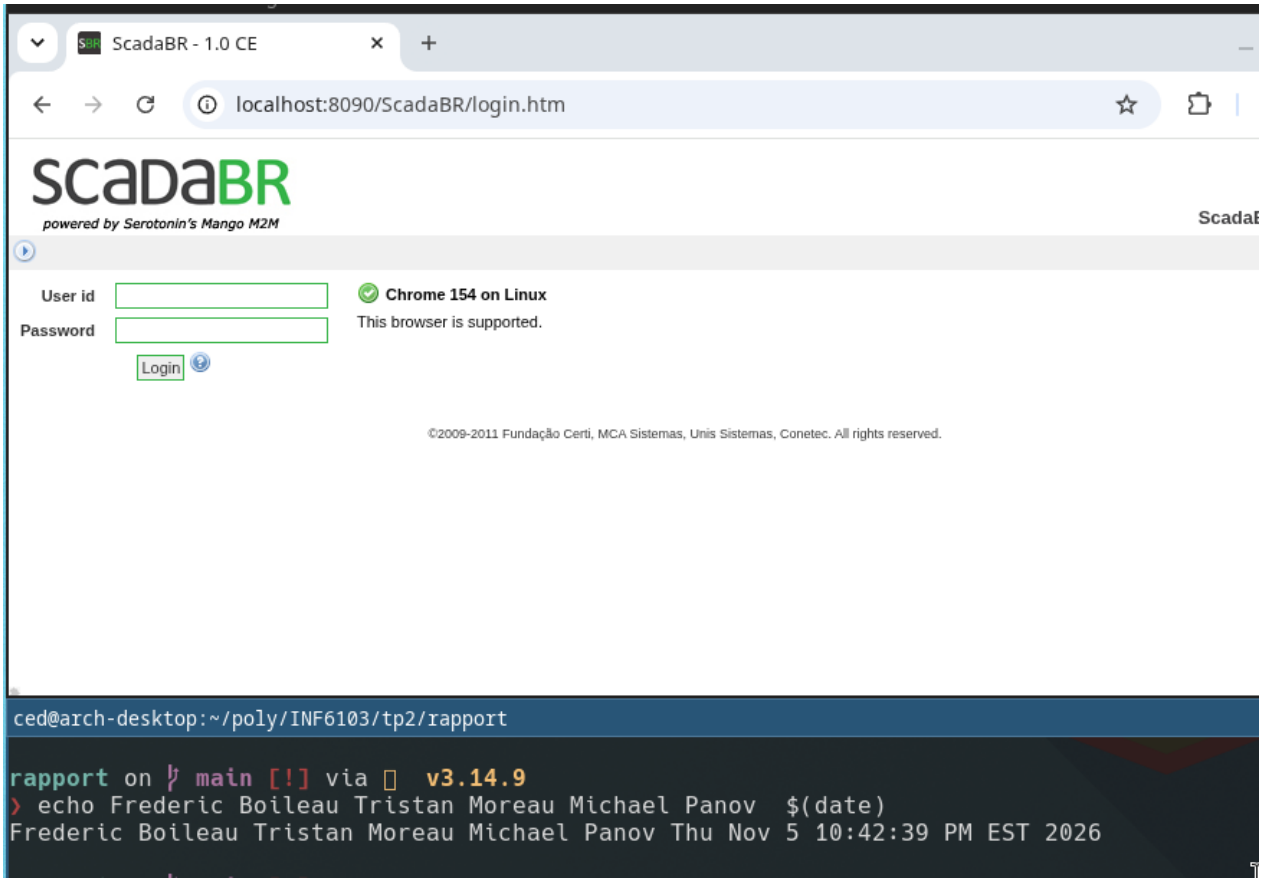


FIGURE 2 – Page d'authentification de ScadaBR

1.4 Tentative d'authentification

Question 1.4 — *Tentez une authentification avec les identifiants* `username=admin` *&* `password=admin`.

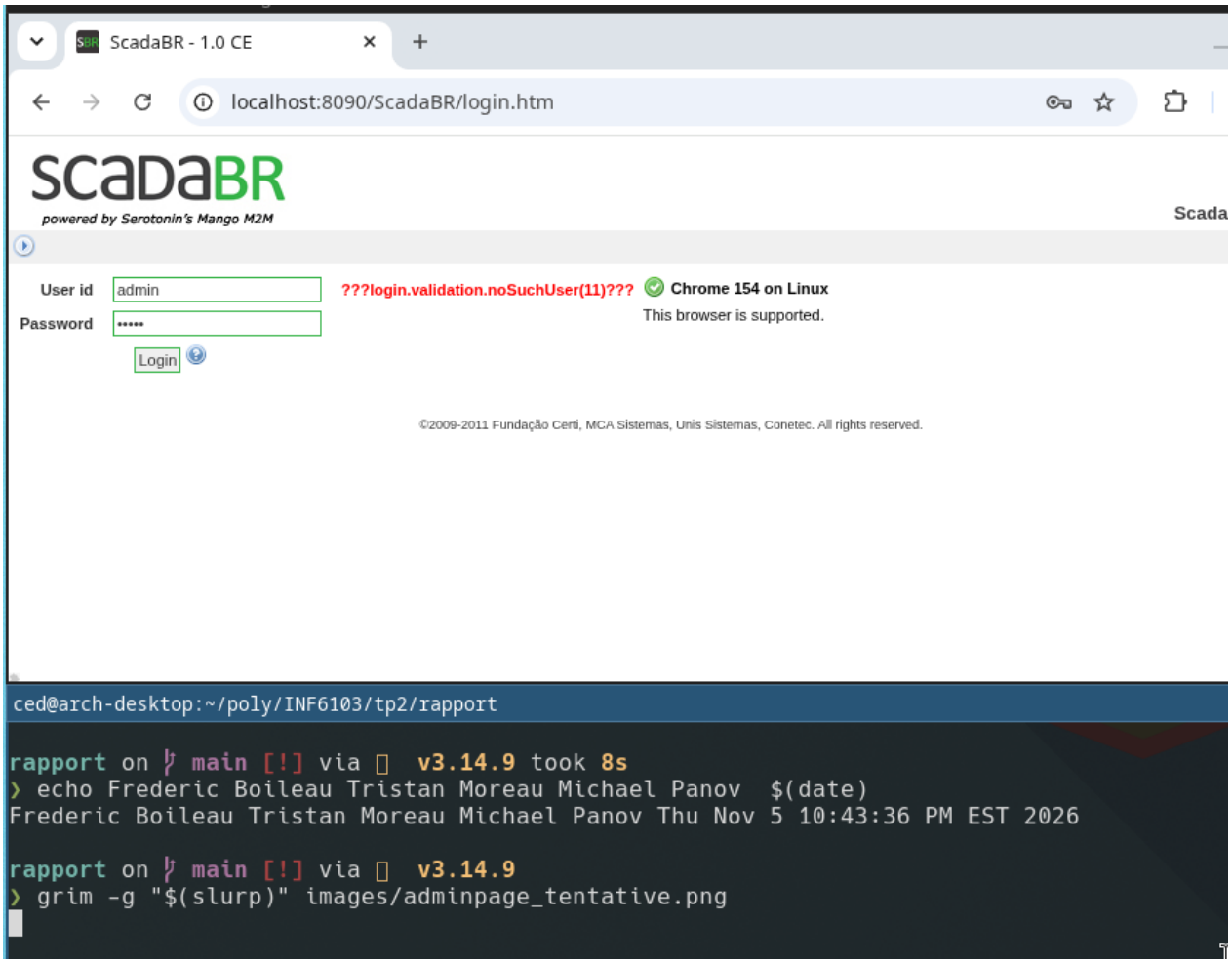


FIGURE 3 – Tentative d'authentification avec `admin`/`admin`

1.5 Brute force

Question 1.5 — *En utilisant les dictionnaires usernames.txt et passwords.txt sur votre RasPi, réalisez une at- taque par force brute sur la page d'authentification pour obtenir les identifiants.*

On utilise le filtre noSuchUser puisque c'est ce que nous voyons en essayant de login avec admin/admin.

```
┌──(root㉿ 2e66cc4cb86d)-[~]
└─#  hydra -L usernames.txt -P passwords.txt  "http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:noSuchUser" -t 64 -I -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-11-06 03:45:48
[WARNING] Restorefile (ignored ...) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 64 tasks per 1 server, overall 64 tasks, 53400 login tries (l:267/p:200), ~835 tries per task
[DATA] attacking http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:noSuchUser
[STATUS] 17236.00 tries/min, 17236 tries in 00:01h, 36164 to do in 00:03h, 64 active
[9090][http-post-form] host: 192.168.95.5   login: massinissa   password: 123456
[STATUS] attack finished for 192.168.95.5 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-11-06 03:47:39
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# echo Frederic Boileau Tristan Moreau Michael Panov $(date)
Frederic Boileau Tristan Moreau Michael Panov Fri Nov 6 03:48:05 UTC 2026
┌──(root㉿ 2e66cc4cb86d)-[~]
```

FIGURE 4 – Brute force username et password

Le message d'erreur change une fois que l'on a trouvé un bon username, nous utilisons donc le filtre Invalid.

```
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# echo Frederic Boileau Tristan Moreau Michael Panov $(date)
Frederic Boileau Tristan Moreau Michael Panov Fri Nov 6 03:52:39 UTC 2026
┌──(root㉿ 2e66cc4cb86d)-[~]
└─# hydra -l massinissa -P passwords.txt  \
> "http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:Invalid" -t 64 -I -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-11-06 03:52:53
[DATA] max 64 tasks per 1 server, overall 64 tasks, 200 login tries (l:1/p:200), ~4 tries per task
[DATA] attacking http-post-form://192.168.95.5:9090/ScadaBR/login.htm:username=^USER^&password=^PASS^:Invalid
[9090][http-post-form] host: 192.168.95.5   login: massinissa   password: q1w2e3r4
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-11-06 03:52:54
┌──(root㉿ 2e66cc4cb86d)-[~]
└─#
```

FIGURE 5 – Brute force avec le bon username

2 Capture du traffic

2.1 Description

Question 2.1 — *À partir de cette interface, décrivez le procédé physique contrôlé : entrées, sorties, capteurs, actionneurs, unités de mesure. Formulez au moins un invariant de sûreté que le PLC devrait maintenir.*

Le procédé physique contrôlé est un réacteur chimique (la cuve centrale) avec deux alimentations en entrée et une mesure de pression et de niveau internes.

Invariant de sûreté : Le PLC doit maintenir la pression du réacteur sous sa limite critique.

Table 2 – Entrées (alimentations) du réacteur

Entrée	Débit observé	Position de vanne
Feed1	$0{,}0\,\mathrm{kmol\,h^{-1}}$	0,0 %
Feed2	$0{,}0\,\mathrm{kmol\,h^{-1}}$	0,0 %

Table 3 – Sorties du réacteur

Sortie	Débit observé	Position de vanne
Purge	$0{,}0\,\mathrm{kmol\,h^{-1}}$	100,0 %
Product	$0{,}0\,\mathrm{kmol\,h^{-1}}$	100,0 %

Capteurs :

- Pression du réacteur : 99.6 kPa
- Niveau : 41.1 %
- Débits sur chaque ligne d'entrée/sortie : kMol/h

Actionneurs

- Vannes réglables sur les deux alimentations et les deux sorties
- Bouton STOP

Unités de mesure

- Débit : kMol/h (kilomoles par heure)
- Pression : kPa (kilopascals)
- Niveau et position de vanne : %

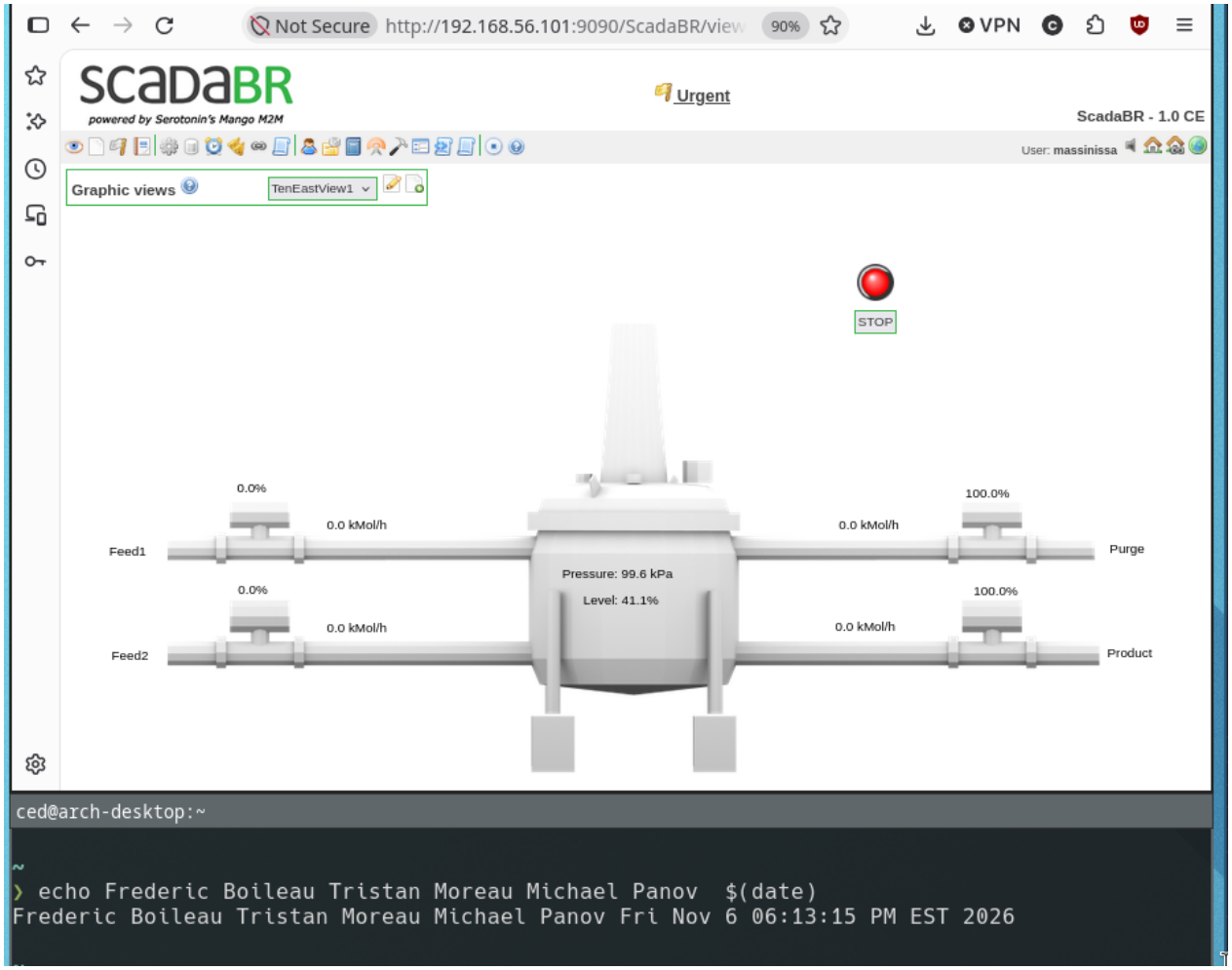


FIGURE 6 – Supervision de ScadaBR

2.2 MiTM

Question 2.2 — *Interposez-vous entre l'IHM et le PLC sans interrompre durablement le procédé.*

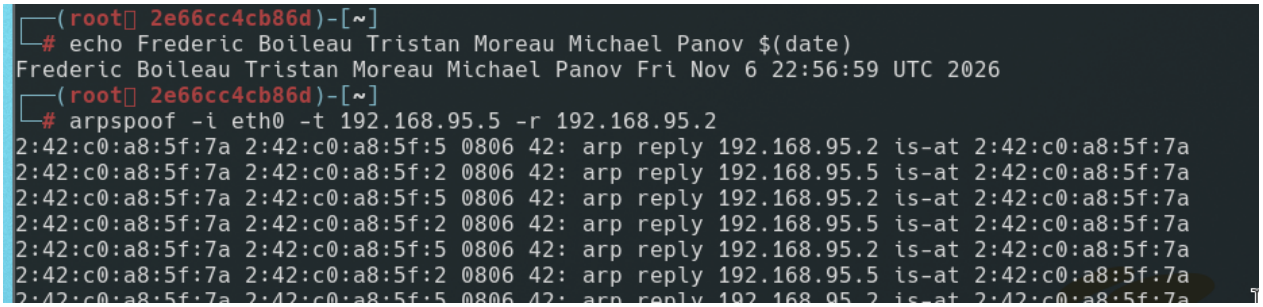


FIGURE 7 – Exécution de arpspoof entre ihm.scada (192.168.95.5) et plc.scada (192.168.95.2)

2.3 Capture et analyse de la communication

Question 2.3 — *Capturez la communication entre les deux et analysez-la.*

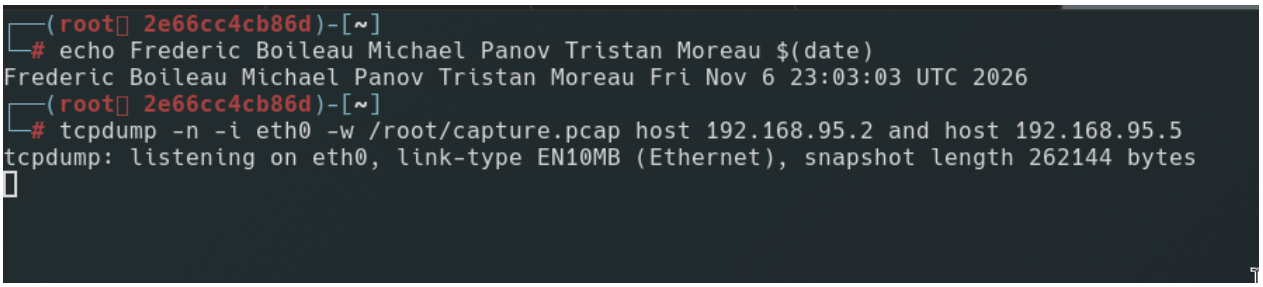


FIGURE 8 – Interception du traffic avec tcpdump

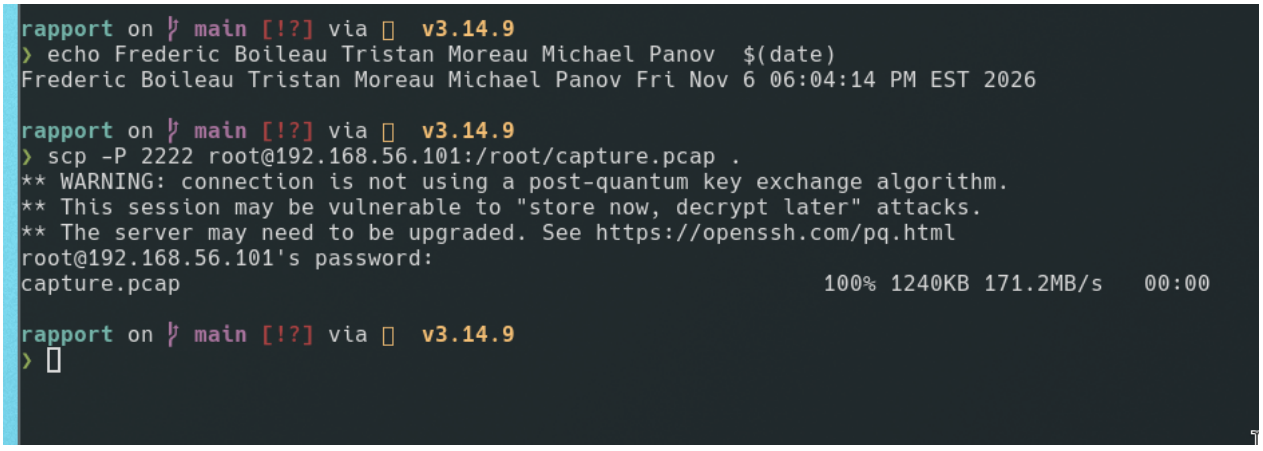


FIGURE 9 – Récupération du fichier pcap

Analyse dans wireshark : Nous filtrons sur le traffic modbus, il y a deux types de messages entre le superviseur et le plc, **read input registers** (13 registres) et **read coils** (un booléen), nous incluons deux captures les montrant. Nous montrons les réponses mais elles sont toutes précédées d'une query. Nous ne voyons que des "read", opérations de lecture passive et aucune écriture. Nous voyons que notre MiTM fonctionne correctement.

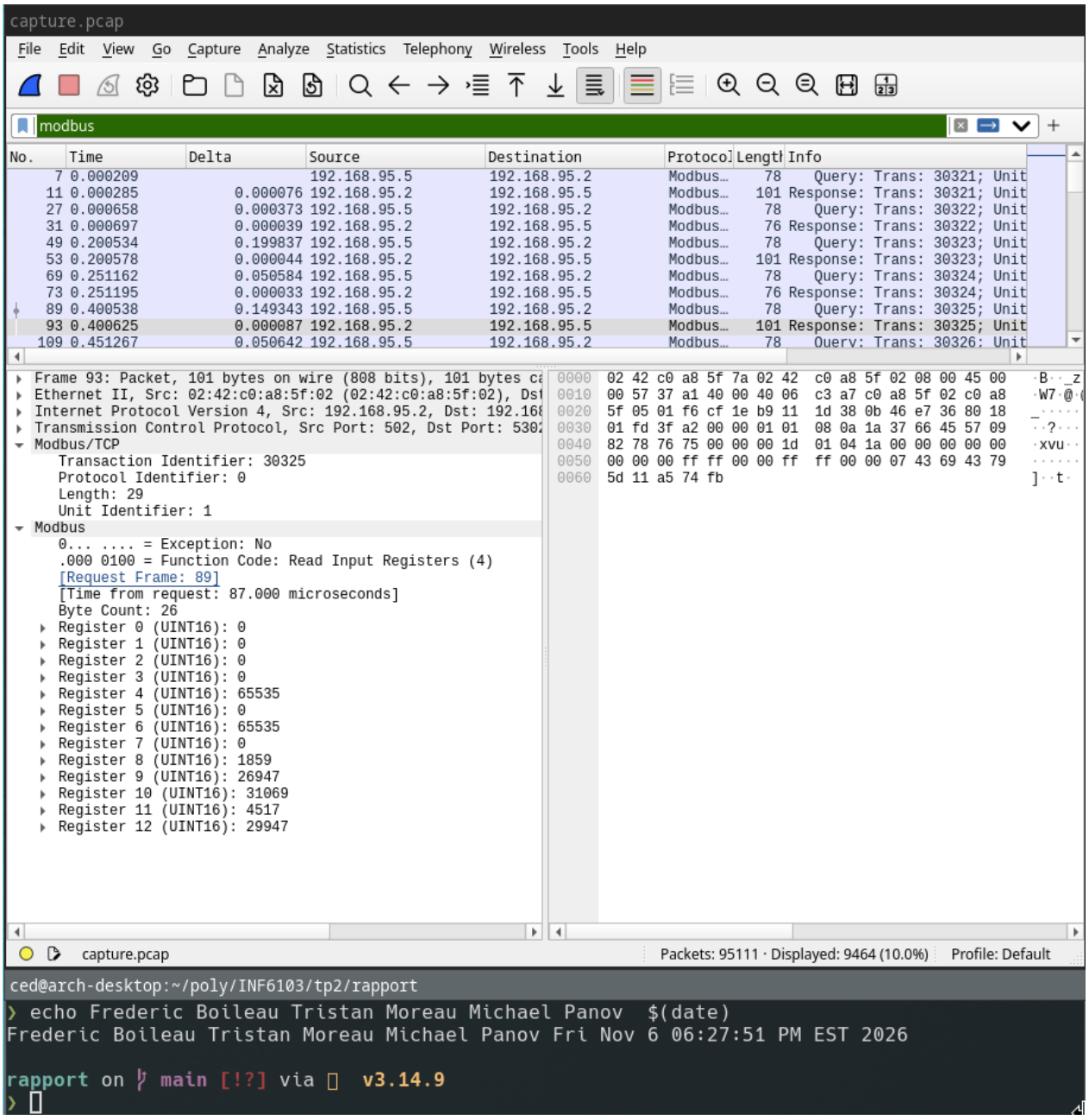


FIGURE 10 – Read input registers

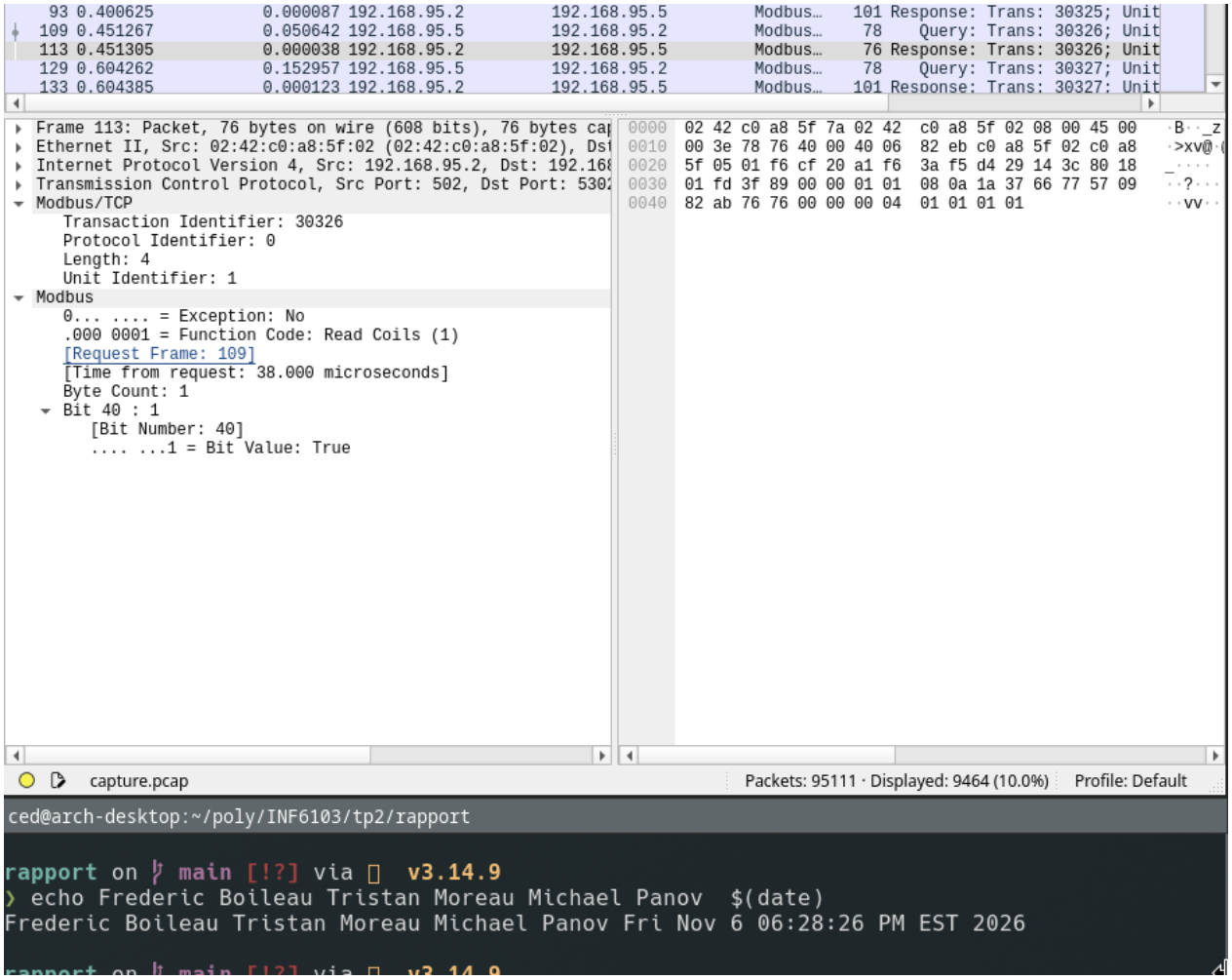


FIGURE 11 – Read coils

2.4 Lecture avec mbpoll

Question 2.4 — *Lisez les valeurs des différents registres du PLC avec votre propre client.*

Nous relançons tcpdump pour la procahine question et lisons les deux registres : celui de 16-bit et celui du coil à l'aide de **mbpoll**.



FIGURE 12 – Lecture des registres avec mbpool avec tcpdump en bg

2.5 Analyse du traffic mbpoll

Question 2.5 — *Capturez le trafic généré par l'action précédente et analysez-le.*

Nous avons maintenant un traffic entre notre machine 192.168.95.122 et le plc 192.168.95.2. À part notre addresse IP le traffic modbus est identique à celui que nous avons capturé précédemment. Nous filtrons encore une fois sur modbus pour ne pas polluer l'interface wireshark avec les handshake TCP.

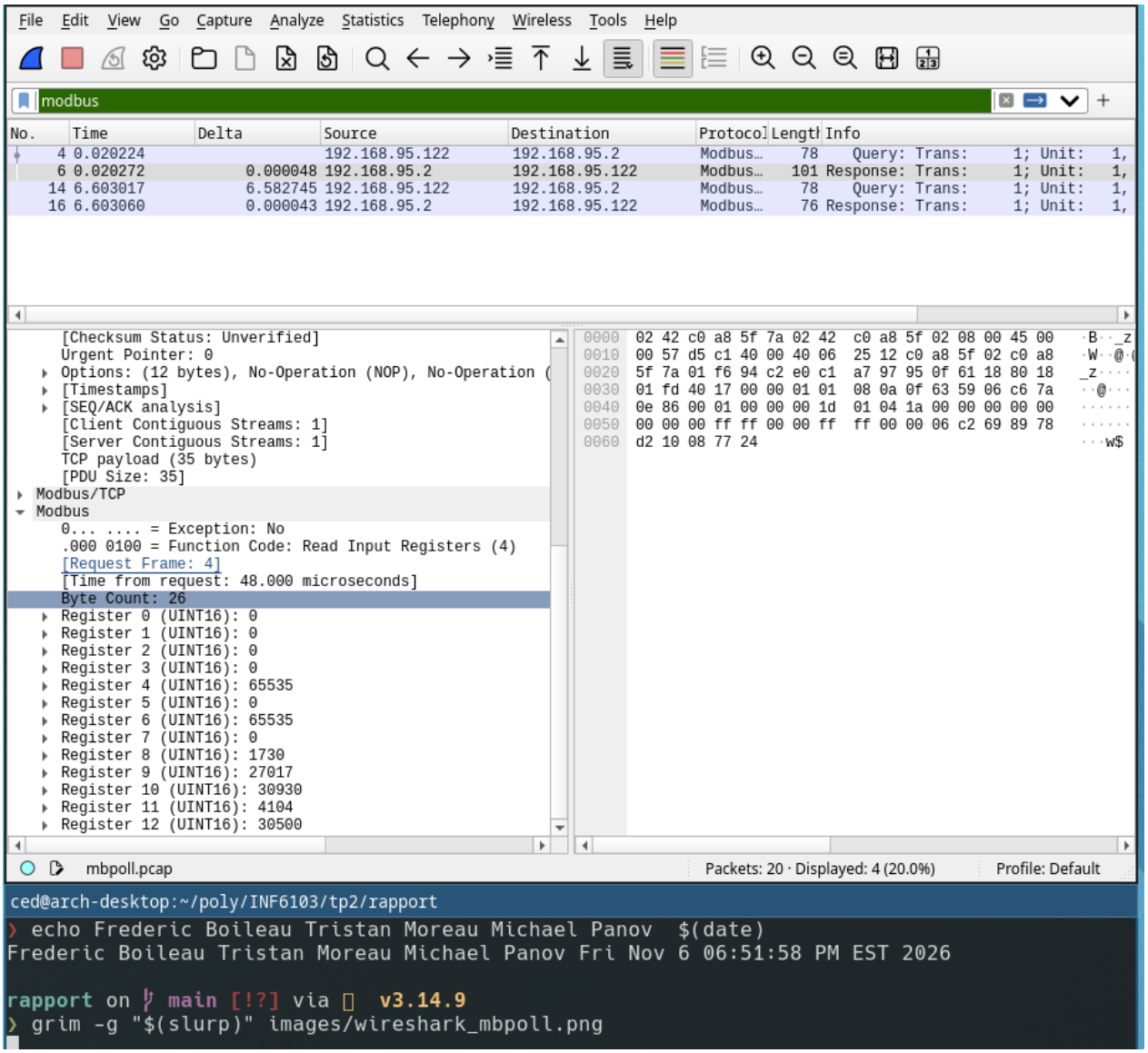


FIGURE 13 – Analyse wireshark mbpoll

3 Injections

3.1 Arrêter le PLC interface

Question 3.1 — *Arrêtez le PLC depuis l'interface de contrôle et identifiez exactement la transaction Modbus responsable. Donnez le code fonction, l'adresse, la valeur écrite et la réponse du PLC. Expliquezpourquoi cette transaction peut être considérée comme critique.*

Nous avons Arrêté le PLC en écoutant le traffic puis avons analysé avec wireshark. Voici les captures de wireshark :

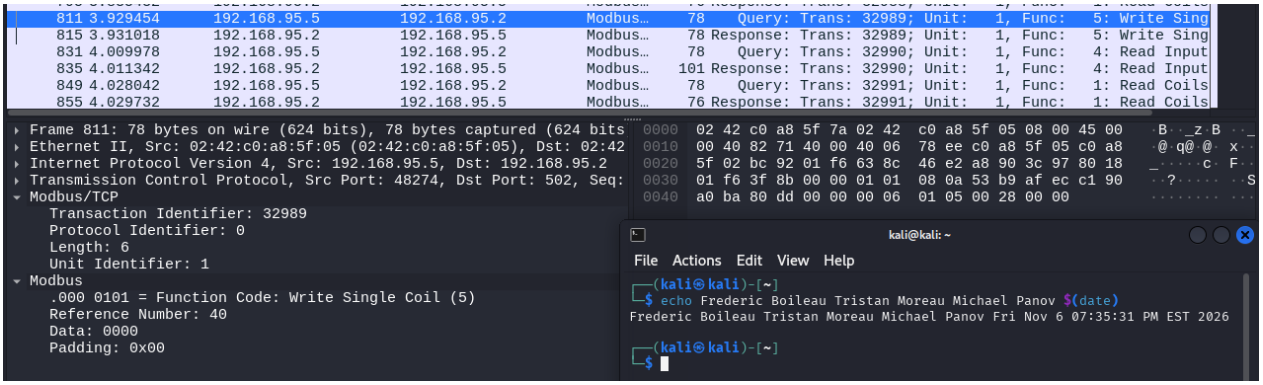


FIGURE 14 – Analyse wireshark modbus query

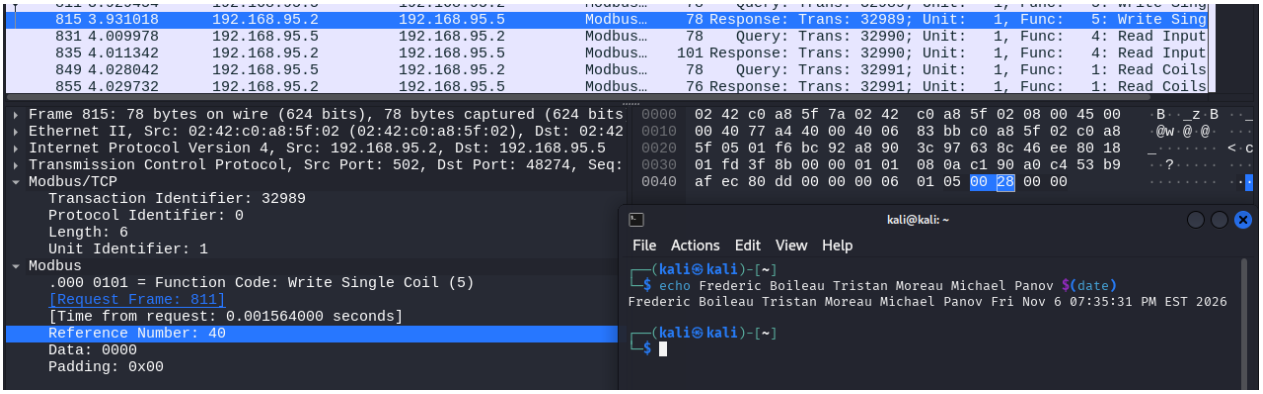


FIGURE 15 – Analyse wireshark modbus response

Le code fonction est Write Single Coil, l'adresse est 40 et la valeur écrite est 00. Comme le veut le protocole modbus, la réponse du PLC est de faire un écho de la demande en cas de réussite, donc le coil a bien été changé.

3.2 Arrêter le PLC commande

Question 3.2 — *Arrêtez le PLC sans passer par une interface graphique.*

En utilisant perl, nous pouvons voir que le PLC s'est bien arrêté :



FIGURE 16 – Avant l'envoi du paquet

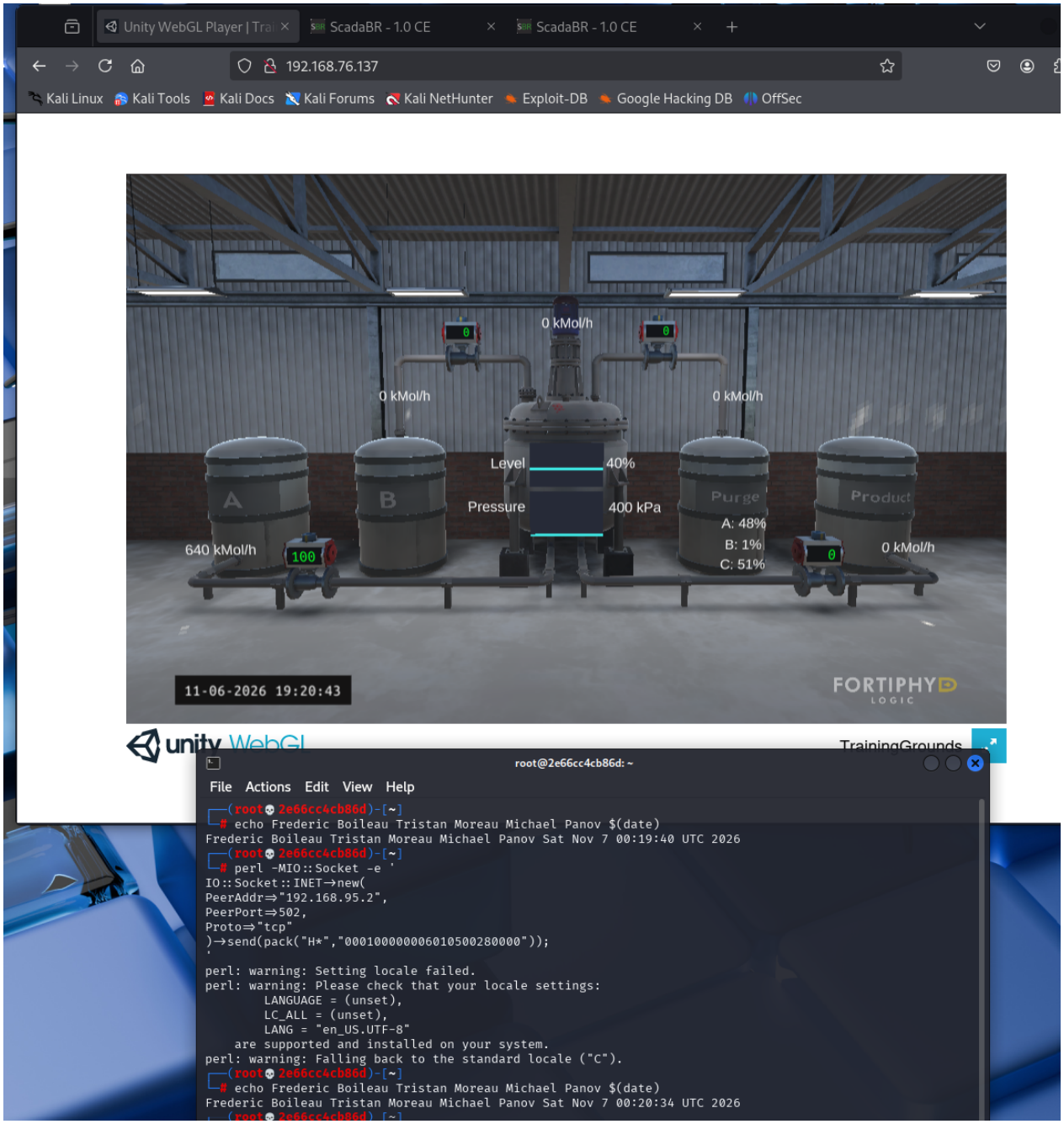


FIGURE 17 – Après l'envoi du paquet

3.3 Modification des valeurs de l'interface de contrôle

Question 3.3 — *Modifiez les valeurs affichées surl'interface de contrôle*

On utilise la commande suivante : **mbpoll -m tcp -a 247 -t 4 -r 1 -0 192.168.95.10 50** suivie de celle-ci :

```
┌──(root💀2e66cc4cb86d)-[/plc]
└─# mbpoll -m tcp -a 247 -t 4 -r 1 -c 2 192.168.95.10
mbpoll 1.0-0 - FieldTalk(tm) Modbus(R) Master Simulator
Copyright © 2015-2019 Pascal JEAN, https://github.com/epsilonrt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

Protocol configuration: Modbus TCP
Slave configuration...: address = [247]
                        start reference = 1, count = 2
Communication.........: 192.168.95.10, port 502, t/o 1.00 s, poll rate 1
000 ms
Data type.............: 16-bit register, output (holding) register table

-- Polling slave 247... Ctrl-C to stop)
[1]:    50
[2]:    0
-- Polling slave 247... Ctrl-C to stop)
[1]:    50
[2]:    0
-- Polling slave 247... Ctrl-C to stop)
[1]:    50
[2]:    0
-- Polling slave 247... Ctrl-C to stop)
```

FIGURE 18 – Valeurs modifiée du flux Feed1

Ces deux commandes permettent de repérer l'esclave Modbus à modifier (ici, le Feed1 dont on trouve l'ID dans le fichier /plc/mbconfig.cfg. On force la valeur de son registre à 50 (donc un flux entrant de 50On voit bien qu'on a réussi à écrire dans le registre de l'esclave (la 2e commande affiche régulièrement les valeurs du Feed1, et on observe que celles-ci sont 50

3.4 Analyse des composants

Question 3.4 — *Analysez les composants du PLC.*

Les composants du PLC sont :

- scale_to_real : composant qui convertit une valeur entière uint en une valeur réelle ;
- scale_to_uint : composant qui convertit une valeur réelle en une valeur entière uint ;
- flow_control : composant qui contrôle le débit de la cuve. Il calcule le débit réel et ajuste la valve d'entrée pour que le débit réel soit celui voulu ;
- composition_control : composant qui contrôle la composition chimique du produit. Si le produit a une concentration trop élevée ou trop faible, il ajuste la valve d'entrée pour réajuster le produit ;
- pressure_control : composant qui contrôle la pression de la cuve. Si la pression est trop élevée ou trop basse, il ajuste la valve de purge pour réajuster la pression ;
- level_control : composant qui contrôle le niveau de liquide de la cuve. Si le niveau de liquide est trop grand ou trop petit, il ajuste la valve de production pour réajuster le niveau de liquide ;
- pressure_override : composant de sécurité qui surveille la pression du système et modifie la consigne de débit lorsque la pression n'est pas comprise dans un seuil prédéfini ;
- initialize_sp : composant d'initialisation exécuté au démarrage du PLC. Il charge les consignes par défaut du système.

3.5 Expérimentation avec différentes valeurs

Question 3.5 — *Expérimentez différentes valeurs objectives (set point) dans le but de faire exploser la cuve.*

Pour faire exploser la cuve, il est le plus judicieux d'augmenter la pression de la cuve au maximum. Cela est possible avec la valeur objective uint press_sp_c dans le composant initialize_sp qui détermine la pression voulue dans la cuve. En mettant cette pression au maximum (65535), la cuve va surement exploser.

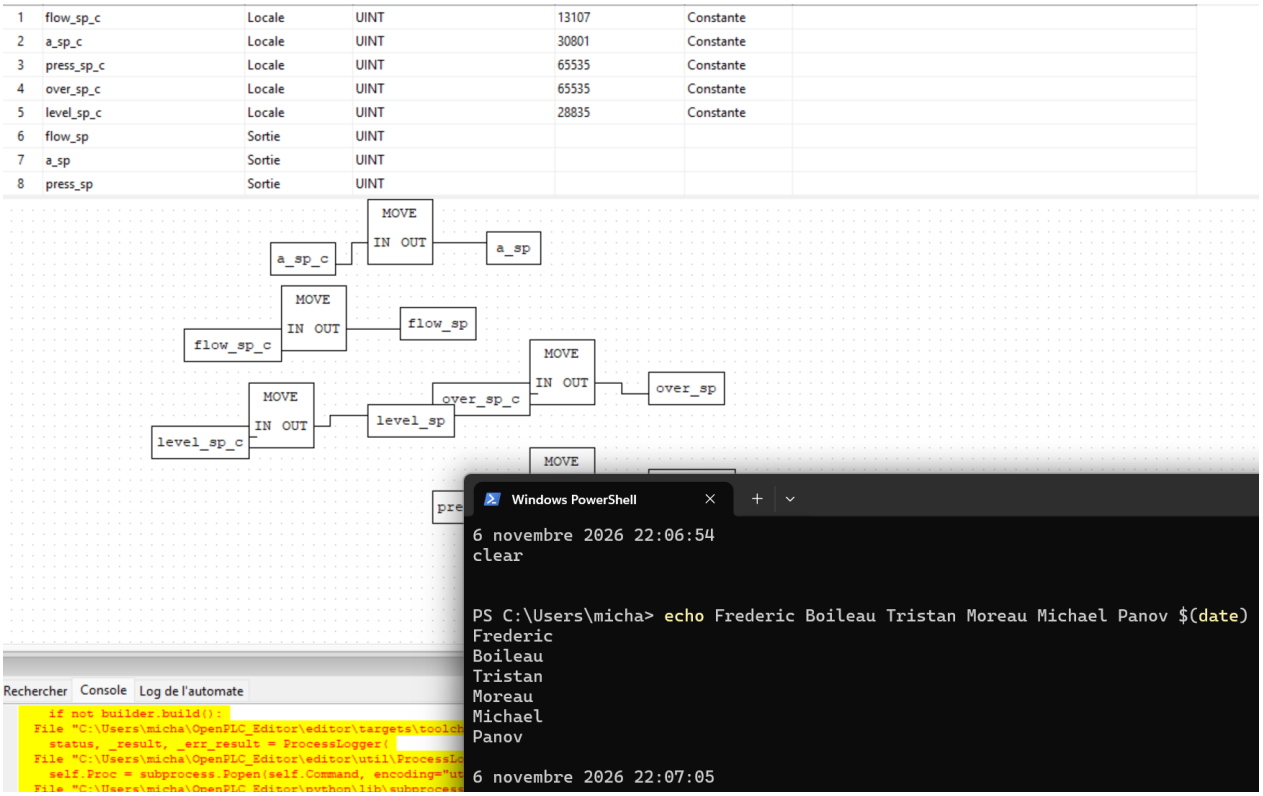


FIGURE 19 – Valeur changée dans OpenPCL Editor

3.6 Explosion

Question 3.6 — *Injectez votre programme malicieux et constatez les dégâts.*

On peut injecter le programme avec le fichier plc.st.



FIGURE 20 – Cuve après l'injection

4 Defense

Les attaques de ce TP viennent d'un même problème : le réseau n'est pas du tout segmenté, ni protégé par des contrôles d'accès. Selon le modèle de Purdue, les niveaux 0 à 2 devraient être isolés par des pare-feux et une DMZ industrielle, ce qui aurait empêché notre RasPi, une fois branchée physiquement dans l'usine, d'accéder directement aux automates et à l'IHM. Des VLANs séparés et un contrôle d'accès réseau rejetant tout appareil inconnu auraient limité notre reconnaissance dès le départ. Sur le plan organisationnel, cette intrusion montre aussi qu'un contrôle d'accès physique plus strict (vérification des intervenants ne faisant pas partie de l'entreprise) est nécessaire, puisque notre scénario reposait entièrement sur une fausse identité de livreur.

Le deuxième problème concerne les mots de passe. Le brute-force a réussi en quelques secondes avec des identifiants faibles (‘massinissa‘/‘q1w2e3r4‘). Il faudrait donc imposer des mots de passe robustes dès l'installation, supprimer les identifiants par défaut, et ajouter une double authentification sur l'interface de supervision. Un verrouillage de compte après plusieurs échecs, combiné à un système de détection d'intrusion capable de repérer un brute-force, aurait aussi bloqué l'attaque avant qu'elle n'aboutisse.

Enfin, les attaques les plus graves, à savoir l'arrêt du PLC et la manipulation des registres — viennent du fait que Modbus/TCP n'authentifie et ne chiffre rien : n'importe qui sur le réseau peut envoyer une commande. Il faudrait un pare-feu industriel qui filtre le trafic Modbus pour n'accepter que les commandes venant de l'IHM, et des limites de sécurité codées directement dans le programme du PLC pour refuser une valeur dangereuse même si la commande semble légitime. L'interface de programmation qui nous a permis d'uploader un programme sans aucune vérification, devrait exiger une authentification forte et n'accepter que des programmes signés. Côté organisation, l'accès à cette interface devrait être réservé à une équipe dédiée, avec un processus de validation avant tout changement et une procédure claire d'arrêt d'urgence.